

CS 2110 Timed Lab 4: C and Dynamic Memory Allocation

Max Everest, Annelise Lloyd, Jonathan Marto, Zaid Akkawi, Sebastian Jankowski

Summer 2023

Contents

1	Timed Lab Rules - Please Read	2
2	Overview	2
2.1	Description	2
3	Instructions	3
3.1	Structs and Global Variables	3
3.2	Writing <code>stack_push()</code>	3
3.3	Writing <code>stack_pop()</code>	4
3.4	Writing <code>destroy_stack()</code>	4
3.5	Testing your program with <code>main()</code>	4
4	Debugging with GDB - List of Commands	5
5	Rubric and Grading	6
5.1	Autograder	6
5.2	Valgrind Errors	6
5.3	Makefile	7
6	Deliverables	7

Please take the time to read the entire document before starting the assignment. It is your responsibility to follow the instructions and rules.

1 Timed Lab Rules - Please Read

You are allowed to submit this timed lab starting from the moment your assignment is released until your individual period is over. You have 75 minutes to complete the lab, unless you have accommodations that have already been discussed with your professor. Gradescope submissions will remain open for several days, but you are not allowed to submit after the lab period is over. **You are responsible for watching your own time. Submitting or resubmitting after your due date may constitute an honor code violation.**

If you have questions during the timed lab, you may ask the TAs for clarification, though you are ultimately responsible for what you submit. The information provided in this Timed Lab document takes precedence. If you notice any conflicting information, please indicate it to your TAs.

The timed lab is open-resource. You may reference your previous homeworks, class notes, etc., but your work must be your own. **No collaboration is allowed for timed labs.** Contact in any form with any other person besides a TA is absolutely forbidden. You are also forbidden from using any materials written with the use of machine learning software such as GitHub Copilot, or ChatGPT.

2 Overview

2.1 Description

2110's Ed Discussion page becomes extremely busy sometimes. After some consultation with the 3510 TAs, the 2110 TAs determined the most efficient method of answering posts was following the first-in, last-out method. In this timed lab, you will implement a stack that will be used to manage Ed posts.

To implement this stack, you will use a array of pointers. That means each array element contains a pointer to an object of type `post`.

In order to help you manage the data structure, we have provided two global pointers. There is `capacity` for the size of the backing array and `numPosts` for the number of items held in the stack. You will be given responsibility to change these variables as needed.

Note that dynamic memory allocation is required for this implementation. You do not know ahead of time how many elements this data structure must hold, so you will have to increase the size of the backing array at compilation.

For this stack, you shall be writing three functions that will aid TAs with keeping track of students on the queue: `push_stack`, `pop_stack`, and `destroy_stack`.

The entire assignment must be done in C. Please read all the directions to avoid confusion.

THERE ARE NO CHECKPOINTS; you can implement the functions in any order you want. Each function can be implemented independently, so you can get full credit for any function without getting credit for any other function. If you think a function is difficult to implement, you can save it for later and work on a different function.

3 Instructions

You have been given three C files - **tl04.c**, **tl04.h**, and **main.c** (**main.c** is only there if you wish to use it for testing; the autograder does not read it). For **tl04.c**, you should implement the **push_stack**, **pop_stack**, and **destroy_stack** functions according to the comments. Optionally, if you want to write your own manual tests for your program, you can implement **main()** in **main.c**. You are allowed to use standard string functions in **string.h** (e.g. **strlen()**, **strncpy()**)

You should not modify tl04.h. Doing so may result in point deductions. You should also **not** modify the **#include** statements nor add any more. You are also not allowed to add any global variables.

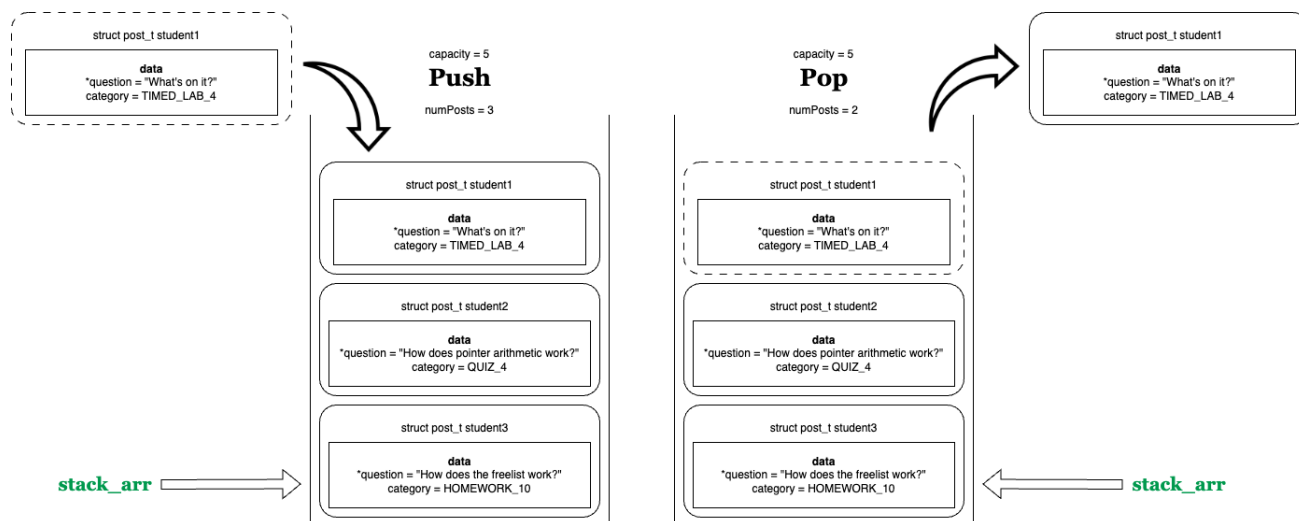
3.1 Structs and Global Variables

This timed lab involves one struct, (**struct post**). This type represents individual post's data. Each post has a string for the question named (**char* question**), as well as the category (**enum category category**) that they need help on. For example, a student might post a question asking "Will there be malloc on Timed Lab 4?" and set the category to "TIMED_LAB_4". **Note: The student's question is stored somewhere else in memory, as indicated in the header files.**

There are three global variables in this file. The variable **stack_arr** keeps track of the bottom of the stack. It is essentially an array containing pointers to **struct post**'s. The **numPosts** variable keeps track of the number of posts on the stack. The **capacity** variable keeps track of the max size of the stack, essentially tracking the max number of posts there is currently space for.

Refer to the following example for a visual representation of the data structure, where "..." denotes an unknown pointer address (pointing to some memory allocated on the heap):

Stack Diagram:



3.2 Writing **stack_push()**

This function will be called by client code to add a post to the top of the stack. The caller will supply the question and category of the post to add. This function should allocate a **post** on the heap, and deep-copy all the data. In particular, any pointers in the **post** will require their own dedicated memory allocation. Make sure that all members of the **post** are set! If the stack is full, double the capacity of the stack and then add the new post to the stack. Finally, insert the post onto the stack with the help of the **stack_arr** pointer. This function should return **SUCCESS** if the post was added successfully. If it fails, it should return **FAILURE** and leave the list unchanged. It should fail if and only if:

- `malloc` fails,
- the post's question is `NULL`, or
- the post's question is an empty string.

Remember to modify `numPosts` and properly insert the post in the stack with the help of the `stack_arr` pointer. Refer back to the diagram for specific details about how the stack works.

3.3 Writing `stack_pop()`

This function will be called by client code to remove a post from the top of the stack. It will return whether a post was removed successfully, and the post removed in that case. The way this function returns the post is using the data out technique. This is to get around the limitation that functions may only have one return value. As such, the caller will pass in a pointer where the post should be stored. Then this function will store the returned post at that pointer. Independently, it returns whether it succeeded via the normal path. First, free the `struct post` being popped and set the pointer equal to `NULL`. Finally, update `numPosts`. Refer back to the PDF/diagram for specific details about how the stack * works, and consider any edge cases.

If this function succeeds, it should return `SUCCESS` and modify `*data` to be the data of the post removed. If it fails, it should return `FAILURE` and leave `arrh` the stack and `*data` unchanged. It should fail if and only if:

- `data` is `NULL`, or
- the stack is empty.

Remember to free any unused data. In this case, the pointer to the question string is still used in `*data` so do not free it!

3.4 Writing `destroy_stack()`

This function will be called by client code to free the whole stack. This involves freeing all elements associated with the stack: every pointer in the stack, every post associated with the stack, and the data associated with each post. Finally, you will set `numPosts` to 0.

When this function succeeds, it should return `'SUCCESS'`.

3.5 Testing your program with `main()`

`main()` can be used to test all of the functions that you've written so far. You can set up your own test cases and check that everything is working.

4 Debugging with GDB - List of Commands

Debug a specific test:

```
$ make run-gdb TEST=test_name
```

Basic Commands:

- `b <function>` **break point** at a specific function
- `b <file>:<line>` **break point** at a specific line number in a file
- `r` **run** your code (be sure to set a break point first)
- `n` **step over** code
- `s` **step into** code
- `p <variable>` **print** variable in current scope (use `p/x` for hexadecimal)
- `bt` **back trace** displays the stack trace

5 Rubric and Grading

5.1 Autograder

We have provided you with a test suite to check your work. You can run these using the Makefile.

Note: There is a file called `test_utils.o` that contains some functions that the test suite needs. We are not providing you the source code for this, so make sure not to accidentally delete this file as you will need to redownload the assignment. This file is not compiled with debugging symbols, so you will not be able to step into it with `gdb` (which will be discussed shortly).

We recommend that you write one function at a time and make sure all of the tests pass before moving on to the next function. Then, you can make sure that you do not have any memory leaks using Valgrind. It doesn't pay to run Valgrind on tests that you haven't passed yet. Below, there are instructions for running Valgrind on an individual test under the Makefile section, as well as how to run it on all of your tests.

The given test cases are the same as the ones on Gradescope. We formally reserve the right to change test cases or weighting after the lab period is over. However, if you pass all the tests and have no memory leaks according to Valgrind, you can be confident that you will get 100% as long as you did not cheat or hard code in values.

Printing out the contents of your structures can't catch all logical and memory errors, which is why we also require you run your code through Valgrind. You will not receive credit for any tests you pass where Valgrind detects memory leaks or memory errors. Gradescope will run Valgrind on your submission, but you may also run the tester locally with Valgrind for ease of use.

We certainly will be checking for memory leaks by using Valgrind, so if you learn how to use it, you'll catch any memory errors before we do.

Your code must not crash, run infinitely, nor generate memory leaks/errors.

Any test we run for which Valgrind reports a memory leak or memory error will receive half or no credit (depending on the test).

If you need help with debugging, there is a C debugger called `gdb` that will help point out problems. See instructions in the Makefile section for running an individual test with `gdb`.

5.2 Valgrind Errors

If you mishandle memory in C, chances are you will lose half or all of a test's credit due to a Valgrind error. You can find a comprehensive guide to Valgrind errors here: <https://valgrind.org/docs/manual/mc-manual.html#mc-manual.errormsgs>

For your convenience, here is a list of common Valgrind errors:

- **Illegal read/write:** this happens when you read or write to memory that was not allocated using `malloc/calloc/realloc`. This can happen if you write to memory that is outside a buffer's bounds, or if you try to use a recently freed pointer. If you have an illegal read/write of 1 byte, then there is likely a string involved; you should make sure that you allocated enough space for all your strings, including the null terminator.
- **Conditional jump or move depends on uninitialized value:** this usually happens if you use `malloc` or `realloc` to allocate memory and forget to initialize the memory. Since `malloc` and `realloc` do not manually clear out memory, you cannot assume that it is full of zeros.
- **Invalid free:** this happens if you free a pointer twice or try to free something that is not heap-allocated. Usually, you won't actually see this error, since it will often cause the program to halt with a `Signal 6 Aborted` message.

- Memory leak: this happens if you forget to free something. The memory leak printout should tell you the location where the leaked data is allocated, so that hopefully gives you an idea of where it was created. Remember that you must free memory if it is not being returned from a function, or if it is not attached to a valid `ascii_image` struct. (Think about what you had to do for `empty_list` in HW9!)

5.3 Makefile

We have provided a Makefile for this timed lab that will build your project. Here are the commands you should be using with this Makefile:

1. To clean your working directory (use this command instead of manually deleting the `.o` files): `make clean`
2. To compile the tests: `make tests`
3. To run all tests at once: `make run-tests`
 - To run a specific test: `make run-tests TEST=test_name`
4. To run all tests at once with Valgrind enabled: `make run-valgrind`
 - To run a specific test with Valgrind enabled: `make run-valgrind TEST=test_name`
5. To debug a specific test using gdb: `make run-gdb TEST=test_name`

Then, at the (gdb) prompt:

- (a) Set some breakpoints (if you need to—for stepping through your code you would, but you wouldn’t if you just want to see where your code is segfaulting) with `b suites/tl4_suite.c:420`, or `b tl04.c:69`, or wherever you want to set a breakpoint
 - (b) Run the test with `run`
 - (c) If you set breakpoints: you can step line-by-line (including into function calls) with `s` or step over function calls with `n`
 - (d) If your code segfaults, you can run `bt` to see a stack trace
6. To compile the code in `main.c`: `make tl04`

To get an individual test name, you can look at the output produced by the tester. For example, the following failed test is `test_set_character_basic`:

```
suites/tl4_suite.c:50:F:test_set_character_basic:test_set_character_basic:0
      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

Beware that segfaulting tests will show the line number of the last test assertion made before the segfault, not the segfaulting line number itself. This is a limitation of the testing library we use. To see what line in your code (or in the tests) is segfaulting, follow the “To debug a specific test using gdb” instructions above.

6 Deliverables

Please upload the following files to Gradescope:

1. `tl04.c`

Your file must compile with our Makefile, which means it must compile with the following gcc flags:

`-std=c99 -pedantic -Wall -Werror -Wextra -Wstrict-prototypes -Wold-style-definition`

All non-compiling timed labs will receive a zero. If you want to avoid this, do not run gcc manually; use the Makefile as described below.

Download and test your submission to make sure you submitted the right files!